Software Engineering with Generative AI

Michigan Technological University — Fall 2026 CS 5090 (3 credits, graduate)

Instructor	Dr. Jie J.W. Wu
Email	jie.jw.wu@mtu.edu
Office hours	Mon/Wed 9:30–10:30am, Rekhi 207 — please make an appointment before visiting
Meeting time / room	Mon/Wed/Fri 12:00 – 12:50pm, Electrical Energy Resources, Room 0227
Semester	Aug 31 – Dec 11, 2026 (no final exam)

MTU Fall 2026 Key Dates

Per the official MTU 2026–27 academic calendar:

Date	Event
Mon, Aug 31	First day of classes (Week 1)
Fri, Sep 18	Project 2 topics circulated — the research thread starts in Week 3
Fri, Oct 2	**Project 2 research abstract due** — context, the gap in the literature, and your proposed solution, in 150–250 words
Wed, Oct 7	**Midterm 1**
Fri, Oct 23	Pooled-findings session in class (bring your current Project 1 findings table)
Mon, Oct 26	Project 2 topic pitches — class held **online**
Wed, Oct 28	Class held **online**
Fri, Oct 30	**No class — deadline day. Project 1 due in full** (one deadline: all four phases + demo video) · **Project 2 proposals due** — topic pivots allowed until Fri, Nov 20
Wed, Nov 18	**Midterm 2** (covers Module 2) — the latest slot MTU exam policy allows a clear run after
Fri, Dec 4	Project 2 full paper drafts due
Mon & Wed, Dec 7–9	**Project 2 design presentations + preliminary results** (in class)
Fri, Dec 11	Last day of classes; **Project 2 final paper due**
Dec 14–18	Finals week — **no final exam in this course**

Holiday class cancellations are noted in the Course Schedule below. Exact midterm days will be adjusted to the actual meeting pattern once scheduled; add/drop and withdrawal deadlines follow the Registrar's calendar.

Course Description

Generative AI — and large language models (LLMs) in particular — is materially changing how software is built. This course examines that change from both directions: **software engineering *of* generative AI systems** (building, evaluating, and operating systems with learned components) and **software engineering *with* generative AI** (AI-assisted and agentic development workflows).

AI-assisted and agentic software engineering is not a standalone unit at the end — it is a **thread that runs through the whole course**. From Week 1, students use AI coding assistants and agents as their required development workflow (and learn to validate their output); the foundations module explains how those tools work by building one; the production module covers how to engineer, evaluate, and operate LLM-powered and agentic systems. The second half of the course runs as a research seminar: students — solo or in pairs — carry out an original research project and write it up as a conference-format paper, which is submitted to a suitable venue after the semester ends.

Prerequisites

- Probability and statistics, linear algebra, data structures
- Programming maturity (Python); prior exposure to machine learning is helpful but not required
- Graduate standing, **or instructor permission for undergraduates**

Undergraduate students

This is a graduate course, and a small number of undergraduates take it with instructor permission. They do the same lectures, the same Project 1, and the same exams. Three adjustments are **recommended — suggestions, not requirements** — so that the graduate-level research expectations land more gently on a student who has not been taught them yet:

	Recommended adjustment
Project 2 pairing	Project 2 is done solo or in pairs; undergraduates are **encouraged to pair with a graduate student** rather than working solo.
Design presentation	Undergraduates paired with a graduate student may **co-present** the design presentation rather than presenting alone.
Reading load	One paper per week is a reasonable target where graduate students read two; the second is optional.

Everything else — Project 1 in full, both midterms, the findings table, the Week 8 pooled-findings session — is identical. Project 1 is deliberately scaffolded phase by phase and does not assume research experience, so it is the right entry point regardless of level.

The workshop-submission bonus is open to undergraduates on the same terms as everyone else.

Learning Outcomes

Upon successful completion of this course, students will be able to:

1. Explain at a principled level how large language models are trained (pretraining, instruction tuning, RLHF/preference optimization) and how they generate output.
2. Build, fine-tune, and evaluate a small code-generation language model end to end, including data preparation, tokenization, training, and benchmark evaluation.
3. Analyze training datasets for provenance, bias, contamination, licensing, and governance implications.
4. Explain embeddings as a representational framework and apply them to retrieval and similarity tasks (e.g., retrieval-augmented generation over codebases).
5. Evaluate AI as a system component with defined cost, latency, reliability, safety, and privacy characteristics, and design systems that mitigate common failure modes (hallucination, adversarial inputs, miscalibration, prompt injection).
6. Apply production ML practices — requirements, architecture, deployment, testing, monitoring, and MLOps — to systems with LLM components.
7. Effectively use AI coding assistants and AI agents within a disciplined, collaborative development workflow (version control, code review, validation of AI-generated code), and design and evaluate agentic systems for SE tasks.
8. Critically read and discuss current research in AI-assisted and agentic software engineering.
9. Design and execute an empirical software engineering study involving generative AI, and communicate the results in a conference-quality research paper.

Course Materials

Two textbooks, used deeply — and no purchase is needed. The free materials listed below fully cover this course. T1 is a Manning book, but its companion repository (the complete, runnable code for every chapter) and the resources under it are enough on their own. **If you are able and want to support T1's author — Sebastian Raschka, a former UW–Madison professor — buying the book is a fine way to do it.** T2 is freely readable online, and every other reading is a research paper available via arXiv or the ACM/IEEE digital libraries (through the MTU library).

Textbooks

#	Text	Used for
T1	S. Raschka, *Build a Large Language Model (From Scratch)*, Manning, 2024	Module 1 and Project 1: tokenization, attention, the GPT architecture, pretraining, fine-tuning — the same build-it path the course takes
T2	C. Kästner, *Machine Learning in Production: From Models to Products*, MIT Press — https://mlip-cmu.github.io/book/	Module 2: requirements, architecture, quality assurance, operations, responsible AI

T1 study materials — free, and fully sufficient for this course:

- Official companion code, chapter by chapter: https://github.com/rasbt/LLMs-from-scratch
- The author's page for the book: https://sebastianraschka.com/llms-from-scratch/

- Community study notes: https://knowledge.zhaoweiguo.com/build/html/x-learning/books/ais/2024/build_llm_from_scratch
- A community re-implementation, notebook by notebook: https://github.com/codewithdark-git/Building-LLMs-from-scratch

Additional books — free, optional

- T. Xiao & J. Zhu, *Foundations of Large Language Models* (2025) — https://arxiv.org/pdf/2501.09223 — free; pre-training, generative models, prompting and alignment in one place. The closest companion to Module 1's second half
- A. E. Hassan et al., *Agentic Software Engineering: A Practical Guide for the AI-Native Engineer* — https://book.agentic-swe.dev/ — the closest companion to the ⚙ thread
- A. Zhang, Z. C. Lipton, M. Li, A. J. Smola, *Dive into Deep Learning* — https://d2l.ai/ — deeper neural-network foundations, with runnable code

The ⚙ AI-assisted / agentic SE thread is grounded directly in the research literature (e.g., Hassan et al.'s agentic-SE roadmap below, and their book above) rather than an assigned textbook — the field moves faster than books do.

Research papers

A **suggested reading list** — mostly software engineering research, with the AI papers needed to make sense of it, and a few of the instructor's own — is published on the course website: **Reading list**. It is a menu, not a required set: each week's readings pair a textbook chapter with 1–2 papers drawn from it, and every slide making a specific claim shows its source at the foot of the slide.

Course Schedule

Two teaching modules with the ⚙ AI-assisted/agentic SE thread woven through both, then a flexible design-review block. Three 50-minute lectures per week (Mon/Wed/Fri) when there is no holiday. ×2 means a topic spans two sessions; ⚙ marks a practice session from the AI-assisted/agentic SE thread — supporting the projects, never examined. Slide PDFs are posted on the course website after each class.

> **Sessions are subject to change.** The week-by-week content below is the plan, not a contract: pacing is adjusted to how the class is actually going, and current events in a fast-moving field sometimes earn a session. Dated deadlines and exam dates change only if announced in class *and* on the course website.
>
> **Holiday cancellations (MTU academic calendar):** no class **Mon, Sep 7** (Labor Day) — Week 2 has **two sessions** — and no classes the week of **Nov 23–27** (Thanksgiving). Marked ⛔ below.

Module 1 — Foundations: Build a Code LM, with the ⚙ thread (Weeks 1–8)

Ten foundations lectures plus three ⚙ practice sessions, each ⚙ session landing just before the Project 1 phase it supports. Slides are posted on the course website after each class.

T1 (Raschka) is read in course order and **finished by Week 5, on purpose** — from Week 7 the readings are research papers, because that is where the field lives. The **one idea** column is what to keep if you keep nothing else.

Week	Dates	Sessions	The one idea	Read	Projects
1	Aug 31 – Sep 4	Course overview · ⚙ **hands-on: assistant, Git, compute, AI-use log** · what a language model is	Tokens, embeddings, next-token prediction	T1 ch. 1–2	**Phase 1 released**
2	Sep 8 – 11	Attention and the Transformer block · position, norms, model families	**Attention and the Transformer block** — the core idea of the course	T1 ch. 3–4 · Vaswani et al. 2017	⛔ Labor Day — two sessions
3	Sep 14 – 18	Decoding, guided output, KV cache, MoE, prompting ×2 · ⚙ **building with AI assistance** — context engineering, decomposition, plan-then-act	Decoding and prompting — the model as an interface you tune at run time	T1 ch. 5 · Chen et al. 2021 (Codex)	training runs start · **P2 topics circulated Fri Sep 18**
4	Sep 21 – 25	Pretraining, data mix, scaling laws, SFT and LoRA ×2 · Project 1 clinic	**Pretraining and scaling** — where capability comes from; LoRA	T1 ch. 5–6 + App. E · Hoffmann et al. 2022 · Hu et al. 2021	Build pace: Sep 25 (recommended)
5	Sep 28 – Oct 2	Preferences, reward models, DPO · reasoning, test-time scaling, GRPO · ⚙ **the new workflow** — author to reviewer, levels of autonomy, the evidence	Instruction following, preferences, and reasoning models	T1 ch. 7 · Ouyang et al. 2022 · Rafailov et al. 2023 · DeepSeek-R1	**P2 abstract due Fri Oct 2** — context, gap, proposed solution
6	Oct 5 – 9	Midterm review · **Midterm 1 (Wed, Oct 7)** · Phase 2 debrief	Revise from each lecture's Key-takeaways slide	—	Use pace: Oct 9 (recommended)
7	Oct 12 – 16	Retrieval over a repo, tool calling, MCP, the agent loop ×2 · ⚙ **agentic SE** — a worked episode, tool design, SWE-bench, failure modes	**Beyond the textbook** — retrieval, tools, the agent loop	Lewis et al. 2020 · Yao et al. 2023 (ReAct) · Jimenez et al. 2024 (SWE-bench)	Break pace: Oct 16 (recommended)
8	Oct 19 – 23	Evaluation: pass@k, benchmarks, contamination · ⚙ **testing with LLMs** — oracle problem, TDD, mutation score · **pooled-findings session** (bring your current table) + module wrap	Evaluating code models honestly	Chen et al. 2021 §2.1 · one leaderboard's contamination statement	Assure pace: Oct 23 (recommended)

Midterm 1 covers the Weeks 1–6 lectures (through reasoning); the Weeks 7–8 material and the ⚙ sessions are not examined on it.

Module 2 — Machine Learning in Production, with the ⚙ thread (Weeks 9–11)

Eight production lectures with three ⚙ pairings — each ⚙ session taught alongside the production topic it extends, selecting from both. Slides are posted on the course website after each class.

T2 (Kästner) is read by chapter cluster, matching the lectures; each ⚙ pairing adds one paper from the slide citations.

Week	Dates	Sessions	The one idea	Read (T2 chapters)	Projects
9	Oct 26 – 30	*Travel week — Mon and Wed held online.* **P2 pitches (Mon, online)** · the model is a small box (Wed, online) · ⛔ **Fri: no class — deadline day**	**The model is a component** — systems thinking around a learned part	From Models to Systems	**P1 due in full + P2 proposal due Fri Oct 30**
10	Nov 2 – 6	When ML is the right tool, goals and measurement · requirements and mistakes **+ ⚙ requirements in the AI era** (paired, select) · architecture, deployment, MLOps	Goals, requirements, **planning for the model being wrong**; architecture	when-to-use-ML and goal-setting · requirements + planning-for-mistakes · architecture/deployment/MLOps	**experiments start Mon Nov 2** — proposal feedback returned
11	Nov 9 – 13	Four-level QA **+ ⚙ assuring generated code** (paired, select) · testing in production, versioning · process, teams, debt **+ ⚙ productivity** and responsible engineering (selections)	**Quality assurance at four levels**; production; teams and responsibility	model-quality + data-quality · testing-in-production · process/teams/debt + responsible-engineering	experiments running · team check-ins in office hours

Midterm 2 covers the Module 2 lectures (taught through Fri Nov 13). The ⚙ pairings are not examined.

Weeks 12–14 — Midterm 2, advanced topics, then the Project 2 finish

The end of the course runs: **exam → advanced SE topics → draft → design presentation with preliminary results → final paper.** The advanced-topic sessions are flexible — they are taught from the ⚙ deep-dive material and current papers, and any of them yields to a Project 2 clinic when teams need it.

Session	Plan
Mon, Nov 16	Midterm 2 review session
Wed, Nov 18	**Midterm 2** — covers the Module 2 decks, taught through Fri Nov 13
Fri, Nov 20	⚙ **Advanced topic 1** — agentic SE, the full deep dive
⛔ Nov 23 – 27	*Thanksgiving — no classes*
Mon, Nov 30	⚙ **Advanced topic 2** — AI for testing and assurance, in depth
Wed, Dec 2	⚙ **Advanced topic 3** — instructor's pick from the current literature, or by class vote
Fri, Dec 4	Paper clinic · **draft due (end of day)**
Mon, Dec 7	**Design presentations + preliminary results**, first block (8 min + 2 questions; five talks per session)
Wed, Dec 9	**Design presentations + preliminary results**, second block
Fri, Dec 11	Course wrap — the professional stance · **final paper + artifact due**

Grading

Component	Weight
Midterm exam 1 (Module 1)	10%
Midterm exam 2 (Module 2)	10%
Participation & in-class engagement	10%
Project 1 — Build it, use it, break it, assure it	25%
Project 2 — Research project (conference submission)	45%
Bonus — workshop submission (instructor-approved)	**+5%**

There is no final exam. The two midterms are short, in-class exams: Midterm 1 covers the Module 1 foundations lectures taught through the Week 6 review (through reasoning — the Weeks 7–8 material on retrieval, agents and evaluation is *not* on it); Midterm 2 covers the Module 2 lectures. The ⚙ AI-assisted/agentic SE sessions are practice-oriented and are assessed through the projects, never on an exam. Each examinable lecture ends with a 'Key takeaways' slide; those slides are the examinable core.

Bonus credit (+5% of the course grade). A team whose final paper is **reviewed and approved by the instructor** submits it to a suitable **conference workshop or similar venue** (e.g., an ICSE/FSE workshop, AIware, or an MSR/EASE track — chosen with the instructor over the winter break; the realistic deadlines fall January–March) and earns bonus credit for every team member. **The target is deliberately a workshop, not a top-tier main track** — a workshop paper of up to **8 pages (not including references)** is what one semester honestly produces, and a good workshop with engaged reviewers teaches more than a main-track rejection. The bonus is awarded for a completed, instructor-approved submission — **not** for acceptance, which is decided long after grades are due. Approval means the paper meets a real venue's bar: a falsifiable research question, a fair baseline, reported variance, an honest threats-to-validity section, and compliance with the venue's formatting and AI-disclosure rules. Teams that are not approved lose nothing; the bonus only ever adds.

Participation & in-class engagement (10%)

- **Sign in, and you have it.** Attendance is the whole requirement — sign the sheet each class. There is no discussion quota and nobody keeps score of how often you speak.
- Discussion is **assumed, not graded**: the paper discussions, the hands-on sessions, the pooled-findings session and the design-presentation Q&A are how this class runs, and everyone is expected to join in naturally.
- **One way to lose points: disrespect.** Every student's questions, answers and presentations are treated with respect. Dismissing or belittling another student's contribution costs participation credit.

Project 1 — Build It, Use It, Break It, Assure It (25%, individual, Weeks 1–9)

Full spec: the Project 1 page on the course website

The question Project 1 answers: *how should software engineers be trained when AI is part of the process?* The course's answer is that you make them build the component, use it for real work, attack it,

and then either engineer the checks that catch what they found **or** improve the model so it stops doing it — the engineer's two honest responses to a defect.

One deadline: everything is due Fri, Oct 30 — one bundle: model card + repo, task report, findings table, QA report + a **3-minute recorded demo video**, AI-use log + reflection. The four phases have a **recommended pace**, not graded deadlines:

Phase	Recommended pace	What you do	Hand in (Oct 30)
1 · Build	by Fri Sep 25	Train a small code model with nanoGPT, then **one ablation** — remove a part, predict the effect, measure it	Model card (incl. ablation) + repo
2 · Use	by Fri Oct 9	Use *your own* model on a real SE task, on 20 real inputs	Task report
3 · Break	by Fri Oct 16	Red-team it — find 10+ distinct failures	**Findings table**
4 · Assure or improve	by Fri Oct 23	Build checks that catch them — **or** change the model so it stops producing them — and measure either way	QA *or* improvement report + demo video

The pace exists because the later phases cannot be compressed into the last weekend — and because the **Week 8 pooled-findings session (Fri, Oct 23)** works from whatever findings table you bring that day. Everyone uses the same eight failure categories, so the class ends the term with a pooled catalogue of how a code model fails and how much of that ordinary testing catches; the final catalogue is assembled from the Oct 30 submissions.

The AI-assisted workflow is **required**, with a log and a one-page reflection.

Graded on what you found and what you built to catch it — not on model quality. A model that produces broken Python is fine; it gives you more to find in Phase 3. Rubric: the Project 1 rubric.

A note on data. Anonymised Phase 3 findings may be used in a study of how this course is taught. Participation is voluntary, opt-in via a Week 1 consent form, and has no bearing on grades in either direction.

Compute: the free GPU tiers (Kaggle, Colab and similar) are the working plan and are enough for this model; an ACCESS classroom allocation has been applied for and will be announced if granted — see the compute guide.

Project 2 — Research Project with Conference Submission (45%, solo or in pairs, Weeks 3–14)

Full spec: the Project 2 page on the course website, with the proposal template, the suggested reading list, and the candidate venues.

An original research project on software engineering with/for generative AI, carried out **solo or in pairs** and written up as a conference-format paper. The goal is to take one question from idea to evidence to a paper an outside reviewer would take seriously — and then actually submit it. The rubric is the same for one student as for two; scope expectations scale with the number of hands, and the proposal is where

scope is agreed. Undergraduates are encouraged to pair with a graduate student (see above — a suggestion, not a requirement).

The research thread starts in Week 3. Topics circulate on **Fri, Sep 18**, and by **Fri, Oct 2** each student writes their paper's **abstract — before doing the research**: the context, the problem the literature leaves open (naming the papers), the proposed solution, and the claim they expect to defend, in 150–250 words. Writing it forces a context, a gap and a plan into one paragraph that either holds together or visibly does not — an evening's work in October instead of a lost month in November. Your Phase 3 findings table will later sharpen it (reread the abstract after red-teaming — several of the best questions come from there). Full-time execution begins Nov 2, giving **five working weeks** before the draft. Many projects extend the Project 1 experience directly — four of the circulated candidate topics reuse the class's own models, probes, and QA checks.

Feedback at every step. Every Project 2 deliverable — abstract, pitch, proposal, draft, presentation — comes back with written feedback aimed at what to do next, while there is still time to act on it. Students are expected to take it seriously; they are also **free to keep their own direction, provided they give the reason** for setting a comment aside. Reasoned disagreement is how research works and is welcome here; quietly repeating the same weakness is not.

Why we submit to a real venue. Writing for an external audience is the pedagogy, not a trophy hunt. A real deadline and real reviewers force the things this course is about: a question you can actually answer, an honest baseline, results you can defend, and limitations you state yourself. Teams that make it that far learn more than any term paper could teach.

How submission works — and what the target is. The target venue is a **conference workshop** (ICSE/FSE workshops, AIware, MSR/EASE tracks and similar), on purpose: a workshop paper of up to 8 pages (not including references) is the right-sized ambition for one semester, and setting that expectation up front is healthier than chasing a top-tier main track. Papers are **submission-ready on Dec 11**; teams may post an arXiv preprint then. Over the winter break the instructor works with each team to pick the workshop whose scope and deadline fit their work — options are discussed in Week 12, and the deadlines that suit a December finish fall between January and March. **Papers the instructor reviews and approves are submitted for +5% bonus credit** (see Grading).

Grading is independent of the outcome. Your grade reflects the quality of the research and the writing — the study you designed, the evidence you gathered, the honesty of your analysis. Acceptance decisions arrive months after grades are final and play no part in them. A rigorous negative result earns full marks.

Milestones (weights are of the Project 2 grade):

Date	Milestone	Weight
Fri, Sep 18	Topics circulated (posted on the course website) — the research thread opens	—
Fri, Oct 2	**Research abstract** — context, gap, proposed solution, expected claim (150–250 words)	5
Mon, Oct 26	Five-minute topic pitch, in class	5
Fri, Oct 30	Written proposal (problem, method, baseline, evaluation plan) — topic pivots allowed until Fri, Nov 20	15

Date	Milestone	Weight
Nov 2 – Dec 4	Execution — five working weeks; the ⚙ advanced sessions run alongside; check-ins in office hours	—
Fri, Dec 4	Full paper draft	15
Dec 7 / Dec 9	Design presentation + preliminary results	15
Fri, Dec 11	Final paper + artifact, submission-ready	45
after	Instructor review over break → workshop chosen with each team; approved papers submitted	+5% course bonus

---

Course Policies

Use of AI tools

AI assistants (Copilot, Claude, ChatGPT, agents, etc.) are **required in Project 1**, where using them under discipline — and validating what they produce — is an explicit learning outcome, logged and reflected on. **Elsewhere in the course they are permitted, subject to the conditions below.** Permitted is not the same as recommended: some of what this course asks you to learn is learned only by doing it yourself, and you are the one who has to judge when a tool is helping you and when it is standing in for the understanding you are here to build.

Conditions, in all cases:

- **You are accountable** for everything you submit. AI-generated code and text must be reviewed, tested, and understood well enough that you can defend every line without the tool in the room. "The AI wrote it" is not a defence for work that is plagiarised, incorrect, insecure, or fabricated — including invented citations, which are treated as fabrication.
- **Disclose** AI use in every deliverable: which tools, for what, and how you validated the output. Project 1 additionally requires an AI-use log. Undisclosed use is an academic-integrity matter, not a style preference.
- **Not permitted during the two midterm exams.**
- **For Project 2, follow the target venue's policy** on AI-generated text — read it before you write, as some venues restrict or forbid it.

Late work

Project milestones lose 10% per day late (max 3 days) unless arranged in advance. The Dec 4 draft and Dec 11 final deadlines are firm because the presentation sessions and the submission timeline depend on them.

Academic integrity

All work is governed by the MTU Academic Integrity Policy. Collaboration within teams is expected; representing others' (or undisclosed AI) work as your own analysis is not.

Accessibility and wellbeing

MTU complies with the ADA and Section 504. Students with disabilities should contact Student Accessibility Services and the instructor early in the semester. Standard MTU syllabus statements (accessibility, non-discrimination, Title IX, mental health resources) apply and will be linked on Canvas.
